

Project 03 예약 SaaS — Phase 0 학습정리

클라우드 배포 파이프라인 먼저 뚫기 (빈 API → Docker → GitHub → Azure → CI/CD)

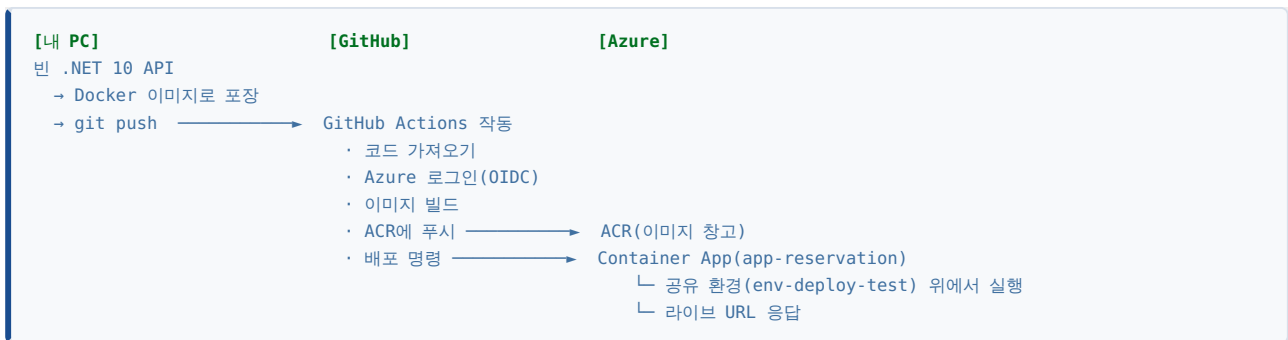
0. Phase 0는 무엇이고, 왜 제일 먼저 했나

목표 한 줄: 기능 코드는 0줄. 오직 `/health` 하나만 응답하는 빈 API를 만들어서, `git push` 한 번에 클라우드에 자동 배포되는 파이프라인을 먼저 깎는다.

왜 배포를 마지막이 아니라 맨 처음에 하나? 보통은 기능을 다 만들고 마지막에 배포한다. 그런데 그러면 "배포에서 막히는 문제"가 프로젝트 끝물에 한꺼번에 터진다. 우리는 반대로 갔다 — 아무 기능 없는 껍데기를 먼저 배포 라인에 올려서, 그 라인이 도는 걸 확인한 뒤 그 위에 기능을 한 단계씩 얹는다. "안 올라간 프로젝트는 0점"이라는 원칙이다. 이 파이프라인이 완성되면, 이후 Phase 1~4는 코드를 push하기만 하면 알아서 배포된다.

Phase 0 최종 성과물 - 라이브 URL: <https://app-reservation.<환경고유값>.westus2.azurecontainerapps.io/health> → `{"status": "ok"}` - `git push` → 자동 빌드 → 이미지 저장소(ACR) 푸시 → Azure 배포까지 전 과정 자동화

전체 흐름 한눈에



이번 Phase의 인프라 전략 (비용 절약) 무료 구독의 제약과 비용을 고려해, Todo 프로젝트가 쓰던 인프라를 최대한 재사용했다.

자원	처리	이유
ACR (이미지 창고) <code>acrdbl0412</code>	재사용	레지스트리는 하나면 됨. 이미지 이름만 분리
Container Apps 환경 <code>env-deploy-test</code>	재사용 (필수)	무료 구독은 환경이 구독당 1개 제한
OIDC 앱 등록 <code>github-actions-deploy-test</code>	재사용	레포별 출입 규칙·권한만 추가
리소스 그룹 <code>rg-reservation</code>	신규	03 자원만 모아 정리(삭제)하기 쉽게
Container App <code>app-reservation</code>	신규	03 전용 앱

핵심 사고방식: 그릇(ACR·환경)은 공유, 내용물(이미지·앱)은 분리. 이 구조 자체가 "비용 제약 속 인프라 설계"라는 면접 카드가 된다.

1부. 로컬 — 빈 API 만들기

STEP 1. 개발 도구 확인

가장 먼저 4개 도구가 깔려 있는지 확인했다.

```
dotnet --version    # 10.0.301 → .NET 10 SDK
docker --version    # Docker version 29.x
git --version
az --version        # azure-cli 2.86.0
```

각 도구가 뭔지 - **dotnet**: .NET SDK(소프트웨어 개발 키트)를 다루는 명령. 프로젝트 생성·빌드·실행을 담당. **--version** 이 **10.0.x** 면 .NET 10이 깔린 것. - **docker**: 앱을 "컨테이너"라는 격리된 상자로 포장·실행하는 도구. - **git**: 코드 변경 이력을 관리하는 버전 관리 도구(내 PC에서 작동). - **az**: Azure CLI. Azure 클라우드를 명령어로 조작하는 도구.

용어 — SDK vs 런타임: SDK는 "개발에 필요한 모든 것"(컴파일러 + 도구 + 런타임)이고, 런타임은 "실행에만 필요한 것"이다. 내 PC엔 SDK가 깔려 있고, 나중에 만들 컨테이너엔 가벼운 런타임만 넣는다(2부 참고).

왜 .NET 9가 아니라 10인가: .NET 9는 STS(단기 지원)라 2026년 11월에 지원이 끝난다. .NET 10은 LTS(장기 지원)로 2028년 11월까지 지원되고, VS 2026이 네이티브로 타깃한다. 아직용 포트폴리오에선 "회사들이 지금 마이그레이션해 가는 최신 LTS"라는 시그널이 중요하다. (단, EF Core·Npgsql 등 패키지도 전부 10 버전으로 통일해야 충돌이 없다 — Phase 1에서 적용.)

STEP 2. 빈 GitHub 레포 생성

reservation-saas 라는 이름으로, README·.gitignore·license를 **모두 체크 해제**해서 완전히 빈 레포로 만들었다.

왜 비워야 하나: 곧 내 PC에서 만든 프로젝트를 이 레포에 처음 올릴(push) 건데, 레포에 파일이 미리 들어 있으면 "로컬엔 없고 원격엔 있는" 상태가 되어 첫 push가 충돌로 거부된다. 빈 방이어야 내 짐을 깔끔히 들여놓을 수 있다.

STEP 3. 빈 API 프로젝트 생성

Visual Studio 2026의 GUI로 "ASP.NET Core 웹 API" 템플릿을 골라 생성했다. (CLI로 하면 아래 세 줄과 같다.)

```
dotnet new sln -n Reservation_Saas    # 솔루션(프로젝트 담는 바구니) 생성
dotnet new webapi -n Reservation_API   # API 프로젝트 생성
dotnet sln add Reservation_API        # 프로젝트를 솔루션에 등록
```

명령어 단어 풀이 - **dotnet new** : 새 무언가를 만든다. - **sln** : solution(솔루션). 여러 프로젝트를 하나로 묶는 **바구니**. 지금은 API 하나뿐이지만, Phase 1에서 클린 아키텍처로 Domain·Application·Infrastructure 프로젝트가 추가되면 이 바구니가 그것들을 묶는다. - **webapi** : Web API 프로젝트 템플릿. RESTful API의 기본 뼈대를 깔아준다. - **-n** : name(이름). 뒤에 오는 이름으로 만든다. - **sln add** : 만든 프로젝트를 솔루션 바구니에 넣는다.

생성 시 고른 옵션과 그 이유

옵션	선택	이유
프레임워크	.NET 10.0	LTS, VS 2026 네이티브
인증 유형	없음	Phase 0엔 인증 불필요(Phase 1에서)
HTTPS 구성	해제	컨테이너는 평문 HTTP만 사용. TLS는 Azure가 처리(2부)
컨테이너 지원	해제	Dockerfile은 직접 작성(VS 자동생성본은 우리 설정과 안 맞음)
컨트롤러 사용	해제	/health 하나엔 최소 API가 가벼움
OpenAPI 지원	체크	API 문서(Swagger)용

최소 API vs 컨트롤러: 최소 API는 **Program.cs** 한 파일에 라우팅을 다 적는 가벼운 방식. 컨트롤러는 기능별로 클래스 파일을 나누는 방식으로, 엔드포인트가 많아질 때 정리에 유리하다. Phase 0는 엔드포인트 1개라 최소 API, Phase 1부터 인증·예약 CRUD로 엔드포인트가 늘면 컨트롤러로 전환한다. **"규모에 맞춰 도구를 고른다"**는 판단 자체가 시니어 시그널이다.

STEP 4. **/health** 엔드포인트 작성

VS가 넣어준 낱씨 예보 샘플 코드를 모두 건어내고, **Program.cs** 에 아래 한 줄을 추가했다.

```
app.MapGet("/health", () => new { status = "ok" });
```

한 줄 풀이 - MapGet : "이 주소로 GET 요청이 오면, 이 함수를 실행하라"고 연결(map)한다. - **"/health"** : 연결할 주소(경로). - **() => new { status = "ok" }** : 실행할 함수. **{ status = "ok" }** 객체를 반환하면, ASP.NET Core가 자동으로 JSON **{ "status": "ok" }** 으로 변환해 응답한다.

왜 /health 인가: 관계적인 "상태 점검(health check)" 엔드포인트다. 모니터링 도구나 클라우드가 "이 서버 살아있나?"를 두드릴 때 쓰는 가벼운 주소. 무거운 로직 없이 "ok"만 답하면 되므로 파이프라인 검증용으로 적합하다.

STEP 5. 포트를 8080으로 통일

Properties/launchSettings.json 의 **applicationUrl** 을 5139(VS가 임의 지정) → **8080**으로 바꿨다.

```
"applicationUrl": "http://localhost:8080",
```

왜 8080인가: 곧 만들 Dockerfile과 Azure 배포 설정이 전부 8080 기준이다. 로컬과 컨테이너의 포트를 8080으로 통일하면 "로컬에선 되는데 배포하면 안 돼"를 예방한다.

launchSettings.json vs appsettings.json (헷갈리기 쉬움) - **launchSettings.json** (**Properties** 폴더 안): **개발 중 띄울 때만** 쓰는 설정. 어느 포트로 띄울지 등. 배포된 컨테이너는 이 파일을 보지 않는다. - **appsettings.json** (프로젝트 루트): **앱 실행 중** 쓰는 설정. 로그 레벨, 나중엔 DB 연결 문자열 등. 그래서 로컬 포트는 전자에서, 컨테이너 포트는 환경변수(2부)에서 따로 정한다.

2부. Docker — 컨테이너로 포장

컨테이너가 왜 필요한가

지금 API는 "내 PC에 .NET 10이 깔려 있어야만" 돈다. **Docker는 실행에 필요한 모든 것을 상자 하나에 담아**, .NET이 없는 컴퓨터나 Azure 서버에서도 똑같이 돌게 만든다. "내 PC에선 되는데 서버에선 안 돼"를 원천 차단한다.

STEP 6. Dockerfile 작성 (멀티스테이지)

Reservation_API 폴더에 **Dockerfile** 을 직접 작성했다.

```
# ==== 1단계: build (작업장) ====
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY Reservation_API.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish

# ==== 2단계: runtime (손님 집) ====
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://*:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "Reservation_API.dll"]
```

Dockerfile 명령어(키워드) 사전 - **FROM** : 어떤 베이스 이미지에서 시작할지. **sdk:10.0** 은 빌드 도구가 다 든 무거운 환경, **aspnet:10.0** 은 실행만 하는 가벼운 환경. - **AS build / AS runtime** : 이 단계에 이름을 붙인다(나중에 **--from=build** 로 참조). 반드시 대문자 **AS** 권장(소문자는 경고). - **WORKDIR** : 컨테이너 안의 작업 폴더를 지정. 이후 명령이 이 폴더 기준으로 실행됨. - **COPY A B** : 파일을 A(내 코드)에서 B(컨테이너 안)로 복사. - **COPY --from=build** : 다른 단계(build)에서 만든 결과물을 가져옴. ← 멀티스테이지의 핵심. - **RUN** : 빌드 중에 실행할 명령(여기선 **dotnet restore**, **dotnet publish**). - **ENV** : 환경변수 설정. - **EXPOSE** : 이 컨테이너가 쓰는 포트를 문서상 명시(표지판 역할). - **ENTRYPOINT** : 컨테이너가 시작될 때 실행할 명령.

dotnet 명령 단어 풀이 - **dotnet restore** : 프로젝트가 의존하는 NuGet 패키지들을 내려받음(복원). - **dotnet publish** : 배포용으로 빌드해 실행 파일을 뽑아냄. - **-c Release** : configuration = Release. 개발용 Debug가 아닌, 최적화된 배포용 빌드. - **-o /app/publish** : output = 결과물을 이 폴더에 출력.

핵심 줄 2개 추가 설명 - **ENV ASPNETCORE_URLS=http://+:8080** : 컨테이너 안의 앱이 들을 주소·포트. **+** 는 "모든 네트워크 주소에서 수신", **:** 8080 이 포트. 이 **+:8080** 형태의 오타를 조심(작업 시 빈출 실수). - **ENTRYPOINT ["dotnet", "Reservation_API.dll"]** : 컨테이너 켜질 때 **dotnet Reservation_API.dll** (=우리 앱)을 실행. DLL 이름은 프로젝트 폴더명(**Reservation_API**)을 따라 정해지므로 한 글자라도 틀리면 컨테이너가 즉시 죽는다.

멀티스테이지를 왜 쓰나 (면접 빈출): 빌드엔 무거운 SDK(수백 MB)가 필요하지만, 실제 실행엔 필요 없다. 1단계(작업장)에서 빌드만 하고, 2단계(손님 집)엔 **빌드 결과물만** 옮겨 가벼운 런타임 위에 올린다. 최종 이미지가 훨씬 작아져 배포가 빠르고 비용이 준다. 비유: 가구를 작업장에서 만들되, 손님 집엔 완성된 가구만 들고고 톱·대패밥은 안 가져간다.

STEP 7. .dockerignore 작성

Dockerfile 과 같은 위치에 **.dockerignore** 를 만들었다.

```
bin/
obj/
.vs/
Dockerfile
.dockerignore
```

왜 필요한가: Dockerfile의 **COPY . .** 이 전체를 복사하는데, 그대로 두면 내 PC(윈도우)에서 빌드된 **bin / obj** 까지 들어가 컨테이너(리눅스)에서 충돌하거나 빌드를 느리게 한다. **.gitignore** 가 "git아 무시해"라면, **.dockerignore** 는 "**Docker야 상자에 넣지 마**"다.

STEP 8. 빌드하고 실행해서 확인

```
docker build -t reservation-app:local .
docker run -p 8080:8080 reservation-app:local
```

docker build 옵션 풀이 - **build** : Dockerfile을 읽어 이미지(상자)를 만든다. - **-t reservation-app:local** : tag(이름표). 이미지에 **reservation-app** 이라는 이름과 **:local** 이라는 버전표를 붙인다. - **맨 끝 .** : "지금 이 폴더를 기준(빌드 컨텍스트)으로 빌드하라". Dockerfile과 코드가 여기 있다는 뜻. 이 점을 빠뜨리기 쉬움.

docker run 옵션 풀이 - **run** : 이미지를 실제로 실행해 컨테이너로 띄운다. - **-p 8080:8080** : publish/port. **-p <내PC포트>:<컨테이너포트>** . "내 PC 8080으로 온 요청을 상자 안 8080으로 연결"한다. 상자는 기본적으로 바깥과 차단돼 있어 이렇게 문을 뚫어야 접속된다.

확인: 새 터미널에서 **(curl http://localhost:8080/health).Content → {"status":"ok"}** . 이때 응답한 것은 **내 PC의 .NET이 아니라 컨테이너 안의 .NET**이다. "상자만 있으면 어디서든 돈다"가 증명된 순간이며, 이 상자를 그대로 Azure에 올리면 거기서도 똑같이 돈다.

PowerShell의 curl 주의: 윈도우 PowerShell에서 **curl** 은 사실 **Invoke-WebRequest** 의 별명이라 상태코드·헤더가 표로 나온다. 본문만 보려면 **(curl ...).Content** 또는 브라우저로 접속.

3부. Git / GitHub — 코드 올리기

STEP 9. .gitignore 생성

```
dotnet new gitignore
```

.NET용 **.gitignore** 를 자동 생성한다. **bin/** , **obj/** , **.vs/** 등 빌드 결과물을 git이 무시하도록 등록한다. 올릴 필요 없는(빌드하면 다시 생기는) 파일을 제외해 저장소를 깔끔하게 유지한다.

STEP 10. commit & push

```
git add .
git commit -m "Phase 0: 빈 API + /health 엔드포인트"
git push -u origin main
```

git 단어 풀이 - **git add .** : 변경된 파일들을 커밋 후보로 **무대에 올린다(staging)**. **.** 은 현재 폴더 전체. **.gitignore** 가 무시하라 한 것은 올라가지 않는다. - **git commit** : 무대에 올린 것을 **하나의 저장 시점으로 기록(찰칵)**. - **-m "..."** : message. 이 저장 시점에 붙이는 메모. "무엇을 했는지" 한 줄로 적는다. - **git push** : 내 PC의 커밋들을 원격 저장소(GitHub)로 밀어 올린다. - **-u origin main** : **-u** 는 "이 브랜치는 origin/main과 짝꿍"이라고 한 번 등록. 이후엔 **git push** 만 쳐도 여기로 간다. 첫 push에만 붙인다. - **origin** : 원격 저장소(GitHub 레포)에 붙인 별명. 매번 긴 주소 대신 **origin** 으로 부른다.

이후 단계(Docker 파일 추가 등)에서는 짝꿍 설정 덕에 **git push** 만으로 올렸다.

4부. Azure — 인프라 준비

STEP 11. 로그인 & 계정 확인

```
az login          # 브라우저로 Azure 로그인
az account show   # 현재 구독 정보 확인
```

az account show 로 구독 이름·ID·테넌트 ID를 확인했다. 이 중 **구독 ID**와 **테넌트 ID**는 나중에 GitHub 시크릿에 들어간다.

용어: 구독(Subscription) = Azure 요금이 청구되는 단위(계정 안의 결제 묶음). **테넌트(Tenant)** = 조직/디렉터리 단위(누가 이 Azure를 소유하나).

STEP 12. 재사용 인프라 점검

만들기 전에, 재사용할 인프라가 실제로 살아있는지 먼저 확인했다.

```
az containerapp env list --query "[].{name:name, rg:resourceGroup, location:location}" -o table
az acr list --query "[].{name:name, rg:resourceGroup, server:loginServer}" -o table
az group list --query "[].{name:name, location:location}" -o table
```

옵션 풀이 - **list** : 해당 종류의 자원 목록을 보여준다. - **--query "..."** : 결과에서 원하는 필드만 골라서 보여준다. 안의 문법은 JMESPath라는 질의 언어. **[].{name:name, rg:resourceGroup}** 은 "각 항목에서 name과 resourceGroup만 뽑아 name/rg로 이름 붙여 표시". - **-o table** : output을 표(table) 형태로. (다른 값: **json**, **tsv**)

확인 결과: 환경 **env-deploy-test** (West US 2), ACR **acrdglee0412** 모두 생존. **rg-reservation** 은 아직 없음(정상).

중요 — 환경 지역 West US 2: 환경이 westus2에 있으므로, 그 위에 올라갈 **app-reservation** 도 **westus2에서 돈다**. 새 RG를 koreacentral로 만들어도 앱은 환경 지역을 따른다. 한국에서 접속 시 약간 멀지만 포트폴리오엔 무관. (이걸 koreacentral로 옮기려면 환경을 지우고 새로 만들어야 하는데, 그러면 Todo 앱까지 죽으므로 안 한다 — "최적화를 일부러 안 하는 트레이드오프 판단"도 면접 카드.)

STEP 13. 새 리소스 그룹 생성

```
az group create --name rg-reservation --location koreacentral
```

옵션 풀이 - **group create** : 리소스 그룹을 만든다. - **--name** : 그룹 이름. - **--location** : 그룹의 메타데이터를 둘 지역.

용어 — 리소스 그룹(Resource Group): Azure 자원을 담는 **폴더**. 프로젝트별로 폴더를 따로 두면, 정리할 때 이 폴더만 통째로 삭제하면 깔끔하다. 성공 신호는 응답의 **"provisioningState": "Succeeded"**.

STEP 14. OIDC — GitHub Actions가 Azure에 로그인하는 방법

왜 OIDC인가: GitHub Actions가 Azure에 배포하려면 신분을 증명해야 한다. 옛 방식은 **비밀번호를 GitHub에 저장**하는 것인데, 유출 위험과 갱신 부담이 있다(집 열쇠를 복사해 맡기는 격). **OIDC**는 비밀번호 없이, "지정한 레포의 지정한 브랜치에서 온 Actions라면 그때그때 일회용 출입증을 발급"하는 방식이다(경비실에서 신원 확인 후 임시 출입증). 유출될 비밀번호가 없어 더 안전하다.

우리는 Todo가 만든 OIDC 앱(`github-actions-deploy-test`)을 **재사용**하고, 두 가지만 추가했다.

(1) 기존 신분증 ID 확인 & 변수 저장

```
az ad app list --query "[?name:displayName, appId:appId]" -o table
$APP_ID = "0e9347fb-...(본인 값)" # 신분증(앱 등록) ID
$SUB = "73edb081-...(본인 값)" # 구독 ID
```

- **ad app** : Azure AD(현 Entra ID)의 **앱 등록**. GitHub Actions가 사용할 "신분증"에 해당.
- **appId** : 그 신분증의 고유 ID(= 나중의 `AZURE_CLIENT_ID`).
- **\$APP_ID = "..."** : PowerShell 변수. 긴 ID를 담아 재사용. (터미널을 닫으면 변수는 사라지므로 다시 설정해야 함.)

(2) federated credential 추가 — "누가 출입 가능한가" PowerShell의 따옴표 문제 때문에 JSON 파일로 만들어 전달했다.

```
@'
{
  "name": "reservation-saas-main",
  "issuer": "https://token.actions.githubusercontent.com",
  "subject": "repo:dglee0412/reservation-saas:ref:refs/heads/main",
  "audiences": ["api://AzureADTokenExchange"]
}
'@ | Set-Content -Encoding UTF8 reservation-fed-cred.json

az ad app federated-credential create --id $APP_ID --parameters reservation-fed-cred.json
```

- **federated credential**: "이 조건을 만족하는 외부 신원은 이 앱으로 로그인 허용"이라는 규칙.
- **subject** : 가장 중요한 줄. `repo:dglee0412/reservation-saas:ref:refs/heads/main` = "reservation-saas 레포의 main 브랜치". 여기 오타가 나면 Actions 로그인도 거부된다.
- **issuer** : 토큰 발급자(GitHub Actions의 고정 주소).
- **audiences** : 토큰 대상(Azure가 받는 고정값).
- **@' ... '@** : PowerShell의 여러 줄 문자열(here-string). 안의 내용을 그대로 파일로 저장.

(3) 권한 부여 — "출입해서 무엇을 할 수 있나"

```
az role assignment create --assignee $APP_ID --role Contributor --scope /subscriptions/$SUB/resourceGroups/rg-reservation
```

- **role assignment** : 역할(권한) 부여.
- **--assignee \$APP_ID** : 누구에게 — 우리 신분증.
- **--role Contributor** : 무슨 권한 — 기여자. 자원을 만들고 수정·배포할 수 있음. (더 센 Owner는 권한까지 남에게 줄 수 있어 위험하므로 안 줌 = 최소 권한.)
- **--scope /subscriptions/\$SUB/resourceGroups/rg-reservation** : 어디까지 — 범위를 딱 **rg-reservation** **폴더로 못박음**. 이 신분증은 03 폴더에서만 권한이 있고 Todo 등 다른 곳은 못 건드린다.

최소 권한 원칙(Least Privilege): "필요한 곳에, 필요한 만큼만" 권한을 준다. `--scope` 로 RG 하나에 가두면, 신분증이 유출돼도 피해가 그 폴더 안으로 한정된다. 이 사고방식은 05 DB모니터링의 "읽기 전용 유저", 03 멀티테넌시의 "테넌트 격리"와 같은 걸 — 일관된 보안 감각으로 어필된다. ACR 푸시 권한은 따로 안 줘도 된다. 이 신분증은 Todo 배포로 이미 같은 ACR에 푸시 권한이 있기 때문.

STEP 15. Container App 생성 (임시 이미지로 자리 잡기)

진짜 우리 이미지는 아직 ACR에 없으므로(첫 배포 때 생김), Microsoft의 임시 helloworld 이미지로 "자리만" 잡았다.

```
$ENV_ID = az containerapp env show --name env-deploy-test --resource-group rg-deploy-test --query id -o tsv
```

```
az containerapp create --name app-reservation --resource-group rg-reservation --environment $ENV_ID --image mcr.microsoft.com/azuredocs/
```

`env show ... --query id -o tsv` 풀이 - 공유 환경의 **전체 리소스 ID**를 끌어와 `$ENV_ID`에 담는다. - `-o tsv` : output을 tsv(탭 구분, 따옴표 없는 순수 텍스트)로. 변수에 깔끔히 담기 위함. - **왜 이름이 아니라 전체 ID인가**: 환경은 `rg-deploy-test`에 있는데 앱은 `rg-reservation`에 만든다. **리소스 그룹 경계를 넘으므로** 이름만으론 못 찾고 전체 주소(ID)가 필요하다.

`containerapp create` 옵션 풀이 - `--name app-reservation` : 만들 앱 이름. - `--resource-group rg-reservation` : 우리 03 폴더에 만든다. - `--environment $ENV_ID` : 공유 환경(발전소) 위에 올린다 = 새 환경을 안 만들고 재사용. - `--image ...helloworld:latest` : 임시 이미지. 진짜 앱은 첫 배포 때 교체. - `--target-port 80` : 이 helloworld가 듣는 포트(80). 우리 앱(8080)은 첫 배포 때 워크플로가 바뀌준다. - `--ingress external` : 외부 인터넷에서 접속 가능하게 문을 연다. 이게 있어야 라이브 URL이 생긴다. - `--query properties.configuration.ingress.fqdn` : 생성 후 앱 주소(URL)만 뽑아서 보여준다.

용어 — ingress(인그레스): 외부 트래픽이 앱으로 들어오는 입구. `external` 이면 공개 URL이 생기고, 여기서 TLS(HTTPS)도 자동 처리된다. **그래서 컨테이너 내부는 평문 HTTP만 써도 되는 것**(앞서 HTTPS 리다이렉트를 끈 이유). `fqdn` = Fully Qualified Domain Name = 전체 도메인 주소.

결과로 나온 URL(`app-reservation.<고유값>.westus2.azurecontainerapps.io`)을 브라우저로 열어 "Your Azure Container Apps app is live"(helloworld) 화면을 확인 = 인프라 생존 확인.

5부. CI/CD — GitHub Actions 자동배포

STEP 16. GitHub 시크릿 등록 (3개)

레포 Settings → Secrets and variables → Actions에 **OIDC ID 3개만** 등록했다.

시크릿 이름	값	의미
<code>AZURE_CLIENT_ID</code>	<code>\$APP_ID</code>	신분증(앱 등록) ID
<code>AZURE_TENANT_ID</code>	테넌트 ID	조직 ID
<code>AZURE_SUBSCRIPTION_ID</code>	구독 ID	결제 단위 ID

왜 이 3개만 시크릿인가: 이걸 "신분 증명용 민감 값"이라 금고(시크릿)에 넣는다. 반면 ACR·RG·앱 이름은 민감하지 않은 단순 이름이라 시크릿이 아니라 워크플로 안에 그대로 적는다. 또 시크릿은 레포별로 따로 보관되므로, Todo와 값이 같아도 새 레포에 다시 등록해야 한다.

STEP 17. 워크플로 작성 (`.github/workflows/deploy.yml`)

```
New-Item -ItemType Directory -Force -Path .github\workflows
```

- `New-Item` : 새 항목 생성(PowerShell). `-ItemType Directory` : 폴더로. `-Force` : 상위 폴더가 없으면 같이 만들고, 있어도 에러 없이 진행. `.github/workflows` 경로는 GitHub이 워크플로를 찾는 **약속된 위치**(다른 곳에 두면 무시됨).

작성한 `deploy.yml` 핵심 구역:

```

on:
  push:
    branches: [main]          # main에 push되면 작동

permissions:
  id-token: write              # OIDC 출입증 발급 권한 (없으면 로그인 거부)
  contents: read

env:                           # 민감하지 않은 이름들(시크릿 아님)
  ACR_NAME: acrdglee0412
  IMAGE_NAME: reservation-app
  RESOURCE_GROUP: rg-reservation
  CONTAINER_APP: app-reservation
  CONTAINER_ENV: env-deploy-test

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest     # 리눅스 가상 머신에서 실행
    steps:
      - uses: actions/checkout@v4          # 1. 코드 가져오기
      - uses: azure/login@v2               # 2. OIDC로 Azure 로그인
        with:
          client-id: ${ secrets.AZURE_CLIENT_ID }
          tenant-id: ${ secrets.AZURE_TENANT_ID }
          subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }
      - run: az acr login --name ${ env.ACR_NAME } # 3. ACR 로그인
      - run: |                               # 4. 이미지 빌드 & 푸시
          docker build -t ${ env.ACR_NAME }.azurecr.io/${ env.IMAGE_NAME }:${ github.sha } -f Reservation_API/Dockerfile Reservati
          docker push ${ env.ACR_NAME }.azurecr.io/${ env.IMAGE_NAME }:${ github.sha }
      - run: az containerapp up --name ${ env.CONTAINER_APP } --resource-group ${ env.RESOURCE_GROUP } --environment ${ env.CONTAIN

```

YAML 용어·문법 풀이 - **on** : 언제 작동할지(트리거). - **permissions: id-token: write** : GitHub Actions가 OIDC 일회용 토큰을 발급받을 권한. **이 줄이 OIDC의 스위치** — 없으면 Azure 로그인이 거부된다. - **env** : 워크플로 전역 변수. **`\${ env.이름 }`** 으로 꺼내 쓴다. - **jobs / steps** : 작업(job)과 그 안의 단계(step)들. - **runs-on: ubuntu-latest** : 이 작업을 돌릴 가상 머신(러너). 리눅스 최신. - **uses** : 남이 만든 액션(재사용 부품)을 가져다 쓴다. **actions/checkout@v4** (코드 체크아웃), **azure/login@v2** (Azure 로그인). - **run** : 셸 명령을 직접 실행. **|** 는 "여러 줄 명령"을 의미. - **`\${ secrets.X }`** : 금고(시크릿)에서 값 꺼내기. - **`\${ github.sha }`** : 이번 커밋의 고유 번호. 이미지 태그로 써서 "어느 커밋이 배포됐나"를 추적.

az containerapp up 을 쓴 이유 - **up** : "없으면 만들고, 있으면 갱신"하는 똑똑한 배포 명령(idempotent). - **--target-port 8080** : 임시 helloworld의 80 포트를 **우리 앱의 8080으로 자동 교체**. 그래서 수동 포트 변경 단계가 필요 없다. - **--registry-server** : 이미지를 가져올 레지스트리(ACR) 주소. - **한 줄로 작성한 이유**: 줄바꿈(\)을 쓰면 윈도우 줄끝문자(CRLF)가 리눅스 러너에서 명령을 깨뜨린다. 그래서 일부러 한 줄.

STEP 18. 발사 — push & 자동배포

```

git add .
git commit -m "Phase 0: GitHub Actions 자동배포 워크플로 추가"
git push

```

push 순간 GitHub Actions가 깨어나 deploy.yml 순서대로(체크아웃 → 로그인 → ACR 로그인 → 빌드/푸시 → 배포) 실행. **첫 실행에 전부 초록볼**로 성공.

확인: 라이브 URL + **/health** → **`\${ "status": "ok" }`** (임시 helloworld가 우리 .NET 앱으로 교체됨). **Phase 0 완주**.

6부. 실제로 겪은 에러와 교훈

증상	원인	해결 / 교훈
unrecognized arguments: --enviroment	--environment 오타(o 빠짐)	"unrecognized arguments" 뒤에 찍힌 단어의 철자부터 의심. 거의 오타
배포 단계에서 이미지 못 찾을	docker push 의 주소를 .azurecr.ig 로 오타	build·push·deploy의 레지스트리 주소가 글자까지 동일한지 비교
(예방됨) 환경 새로 생성 시 실패	무료 구독은 환경 구독당 1개 제한	새로 만들지 말고 기존 환경 재사용
(예방됨) 환경을 이름으로 참조 시 못 찾을	앱과 환경의 RG가 다름(경계 넘음)	이름이 아닌 전체 리소스 ID로 참조
배포된 URL이 westus2 도메인	앱은 환경 지역을 따름	정상. 환경이 westus2라 앱도 westus2
배포 후 OpenAPI 안 뜸	if (IsDevelopment()) 조건 + 컨테이너는 Production 환경	정상·올바른 동작. 운영 환경에선 API 문서를 끄는 게 보안 관례. 보여줄 엔드포인트가 생기는 Phase 2~3에 결지 결정

YAML 들여쓰기 주의: YAML은 들여쓰기가 문법이며 탭이 아니라 스페이스를 써야 한다. 복붙 시 탭이 섞이면 "파싱 에러". 또 JSON 파일을 PowerShell로 만들 때 BOM이 붙으면 az 가 못 읽으므로 -Encoding UTF8 사용.

7부. CLI 용어 빠른 사전 (모아보기)

용어	뜻
SDK / 런타임	SDK=개발 전부(컴파일러+도구+런타임), 런타임=실행만
이미지 / 컨테이너	이미지=실행 환경을 담은 "상자 설계도", 컨테이너=그 상자를 실제로 띄운 것
멀티스테이지	Dockerfile을 build/runtime 단계로 나눠 최종 이미지를 가법게
레지스트리 / ACR	이미지를 보관하는 창고. ACR=Azure Container Registry
리소스 그룹(RG)	Azure 자원을 담는 폴더
Container Apps 환경	컨테이너 앱들이 올라가는 공유 실행 기반(발전소). 무료 구독 1개 제한
ingress	외부 트래픽 입구. external이면 공개 URL + TLS 자동 처리
fqdn	전체 도메인 주소
OIDC	비밀번호 없이 일회용 토큰으로 신원 증명하는 로그인 방식
앱 등록 / SP	GitHub Actions가 쓰는 Azure 신분증(서비스 주체)
federated credential	"이 레포·브랜치에서 온 신원은 허용"이라는 출입 규칙
role / Contributor	권한. Contributor=자원 생성·수정·배포 가능
scope	권한이 미치는 범위(좁힐수록 안전 = 최소 권한)
--query (JMESPath)	결과에서 원하는 필드만 골라내는 질의
-o tsv/table/json	출력 형식(순수텍스트/표/JSON)
github.sha	커밋 고유번호. 이미지 태그로 추적용
CRLF	윈도우 줄끝문자. 리눅스 러너에서 \ 줄바꿈과 충돌 가능

8부. 면접에서 말할 수 있는 포인트 ("끝까지 설명 가능한 것만")

1. **배포를 Phase 0로 앞당긴 이유** — "안 올라간 프로젝트는 0점". 빈 껍데기로 파이프라인을 먼저 검증하고 기능을 었는다.
 2. **복잡도에 맞춘 도구 선택** — `/health` 하나엔 최소 API, 엔드포인트가 느는 Phase 1엔 컨트롤러. 무지성으로 큰 구조를 안 깬다.
 3. **비용 제약 속 인프라 설계** — ACR·환경을 Todo와 공유(그릇 공유), 이미지·앱·권한은 분리(내용물 분리). 무료 구독 환경 1개 제한을 재사용으로 회피.
 4. **OIDC vs 비밀번호** — 저장된 비밀번호 없이 일회용 토큰으로 인증. 유출 표면을 없앤 보안 선택.
 5. **최소 권한 원칙** — 배포 신분증의 권한을 RG 하나로 `--scope` 제한. 05·03의 격리 설계와 일관된 보안 감각.
 6. **멀티스테이지 컨테이너** — 빌드용 SDK와 실행용 런타임 분리로 최종 이미지 경량화.
 7. **TLS 종료 위치 이해** — 컨테이너는 평문 8080, HTTPS는 Azure ingress가 처리. 그래서 앱 내 HTTPS 리다이렉트를 제거(안 그러면 502/무한 리다이렉트).
 8. **안 한 최적화의 근거** — 환경 지역을 koreacentral로 안 옮김. 기존 앱을 깨야 하고 포트폴리오 트래픽엔 효과 미미. "언제 최적화를 안 할지 아는 것"도 판단력.
-

9부. 다음 — Phase 1 예고 (인증 + 멀티테넌시)

여기서 처음으로 **DB와 클린 아키텍처**가 등장한다. - **패키지 전부 .NET 10 패밀리로 통일**: EF Core 10 / Npgsql 10.x / Design 10.x / dotnet-ef 10.x (버전 어긋남 충돌 예방). - **DB**: 공유 PostgreSQL 서버 `pg-shared-dglee` 에 `reservationdb` 추가 + 03 전용 유저(최소 권한). - **실용적 클린 아키텍처** 골격: Domain / Application / Infrastructure / Presentation. - **인증**: Access Token + Refresh Token 분리, Refresh Token은 httpOnly 쿠키(Todo의 localStorage 방식에서 진일보). - **멀티테넌시**: EF Core Global Query Filter로 테넌트별 데이터 자동 격리. - **컨트롤러 전환** + 마이그레이션 자동 적용(`db.Database.Migrate()`).

Phase 0 학습정리 끝. 작성 기준: .NET 10.0.301 / Azure CLI 2.86 / Docker 29.x / VS 2026.